Web Programming

Rodrigo Martins Pagliares

Last update: 10/27/2025

Web Programming

Rodrigo Martins Pagliares

JavaScript
Dynamic client-side scripting

JavaScript first steps

developer.mozilla.org

Get real-time assistance with your coding queries. Try AI Help now!

mdn web docs _ References Guides Plus Blog Play NEW AI Help BETA Theme Log in Get MDN Plus

Guides English (US)

Filter

HTML — Structuring the web

- Introduction to HTML
- Multimedia and embedding
- HTML tables

CSS — Styling the web

- CSS first steps
- CSS building blocks
- Styling text
- CSS layout

JavaScript — Dynamic client-side scripting

- JavaScript first steps
- JavaScript building blocks
- Introducing JavaScript objects
- Asynchronous JavaScript
- Client-side web APIs

Web forms — Working with user data

Learn web development

Welcome to the MDN learning area. This set of articles aims to guide complete beginners to web development with all that they need to start coding websites.

The aim of this area of MDN is not to take you from "beginner" to "expert" but to take you from "beginner" to "comfortable." From there, you should be able to start making your way, learning from the rest of MDN, and other intermediate to advanced resources that assume a lot of previous knowledge.

If you are a complete beginner, web development can be challenging — we will hold your hand and provide enough detail for you to feel comfortable and learn the topics properly. You should feel at home whether you are a student learning web development (on your own or as part of a class), a teacher looking for class materials, a hobbyist, or someone who just wants to understand more about how web technologies work.

Looking to become a front-end web developer?

We have put together a course that includes a
towards your goal.

Get started

In this article

- Where to start
- Topics covered
- Getting our code examples
- Contact us
- See also



How to uplevel your hybrid cloud connection on Equinix Metal using Equinix Fabric

Mozilla ads

https://tinyurl.com/mryx9yzw

What is JavaScript?

Lecture: What is JavaScript?
Module: JavaScript first steps

Filter

JavaScript — Dynamic client-side scripting

- JavaScript first steps
  - JavaScript First Steps
  - What is JavaScript?
  - A first splash into JavaScript
  - What went wrong? Troubleshooting JavaScript
  - Storing the information you need — Variables
  - Basic math in JavaScript — numbers and operators
  - Handling text — strings in JavaScript
  - Useful string methods
  - Arrays
  - Silly story generator
- JavaScript building blocks
- Introducing JavaScript objects
- Asynchronous JavaScript
- Client-side web APIs

Web forms — Working with user

What is JavaScript?

Overview: First steps Next

Welcome to the MDN beginner's JavaScript course! In this article we will look at JavaScript from a high level, answering questions such as "What is it?" and "What can you do with it?", and making sure you are comfortable with JavaScript's purpose.

Prerequisites:	Basic computer literacy, a basic understanding of HTML and CSS.
Objective:	To gain familiarity with what JavaScript is, what it can do, and how it fits into a website.

A high-level definition

JavaScript is a scripting or programming language that allows you to implement complex features on web pages — every time a web page does more than just sit there and display static information for you to look at — displaying timely content updates, interactive maps, animated 2D/3D graphics, scrolling video jukeboxes, etc. — you can bet that JavaScript is probably involved. It is the third layer of the layer cake of standard web technologies, two of which (HTML and CSS) we have covered in much more detail in other parts of the Learning Area.



In this article

A high-level definition
So what can it really do?
What is JavaScript doing on your page?
How do you add JavaScript to your page?
Comments
Summary

GitLab
The One DevOps platform for software innovation. Eliminate point solution sprawl. 30

https://tinyurl.com/2ch4k3wn

Objective of the lecture

Objective: To gain **familiarity** with what **JavaScript** is, what it **can do**, and **how** it **fits** into a **web site**.

Introduction

Introduction

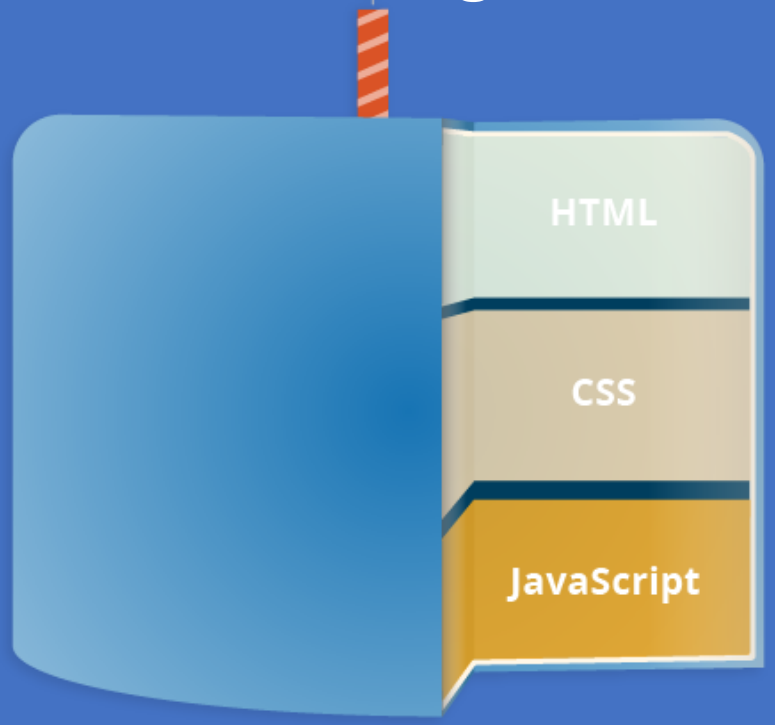
- In this **lecture** we will look at **JavaScript** from a **high level**, answering **questions** such as:
  - ✓ **What is it?**
  - ✓ **What** can **you** do **with it?**
- In addition to **make sure** you are **comfortable** with **JavaScript's purpose**.

A high–level definition

A high–level definition

- **JavaScript** is a **scripting** or **programming language** that allows you to **implement complex features** on **web pages**.

- It is the **third layer** of the **layer cake** of **standard web technologies**.
  - ✓The **first two layers** are **HTML** and **CSS**.



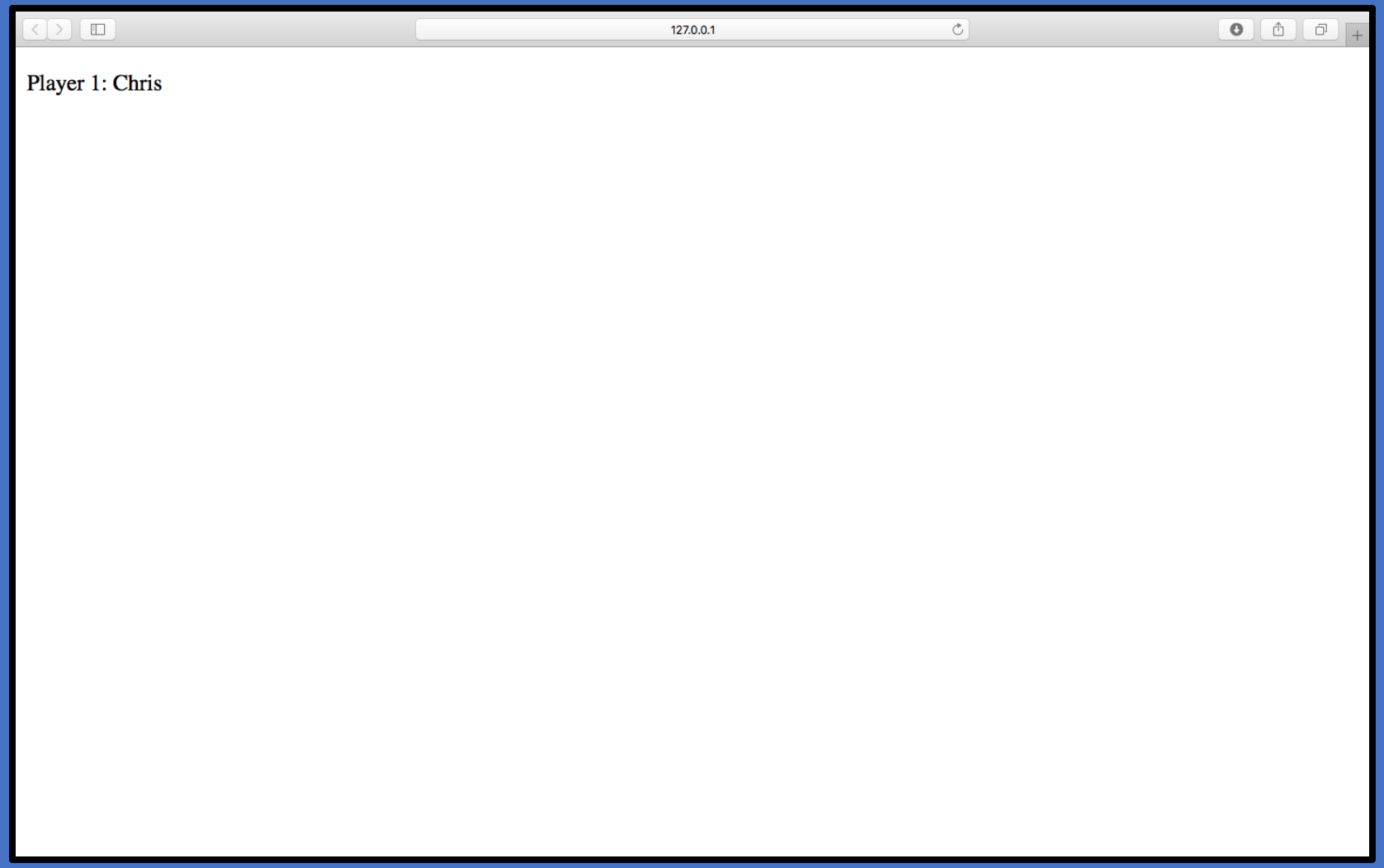
A high–level definition

- **JavaScript enables** you to:
  - ✓Create **dynamically updating content**
  - ✓**Control multimedia**
  - ✓**Animate images**
  - ✓…
  - ✓…

A high–level definition

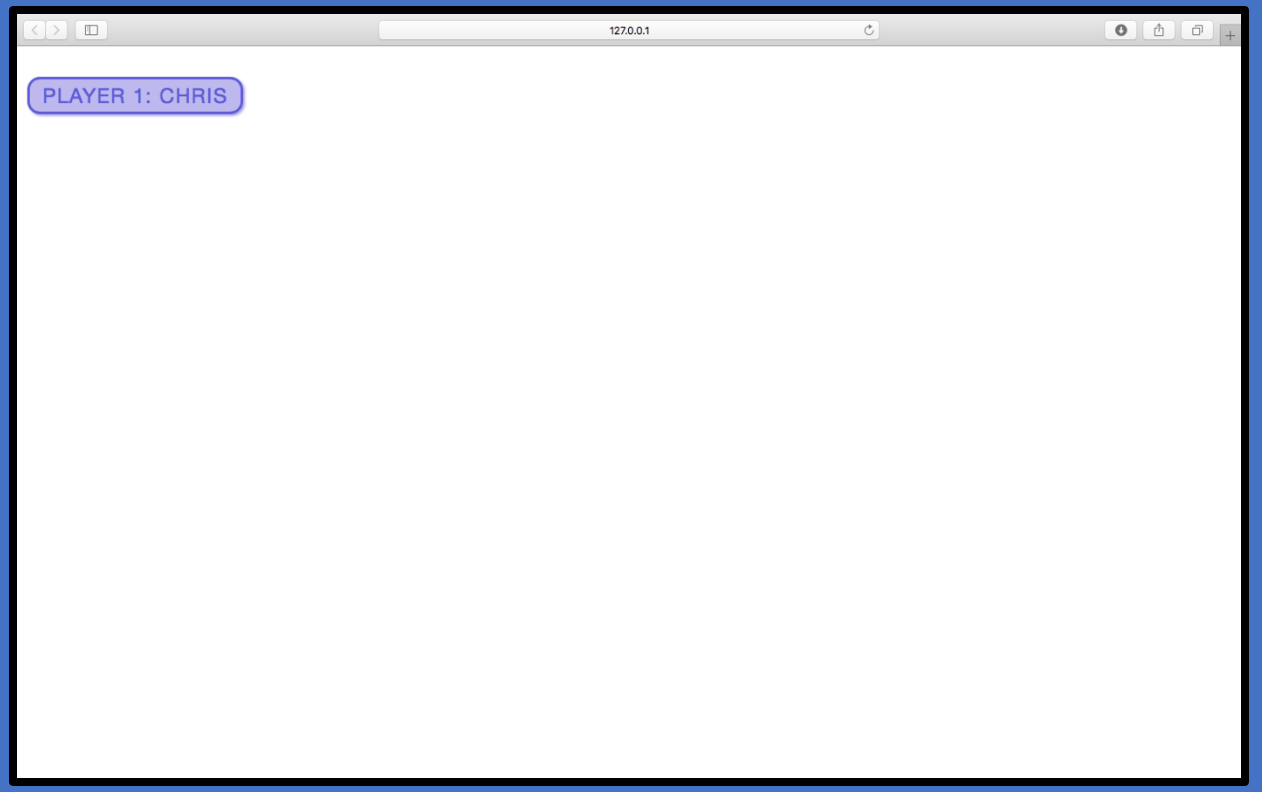
```
1  <p>Player 1: Chris</p>
```

HTML



A high–level definition

```
1  <p>Player 1: Chris</p>
```



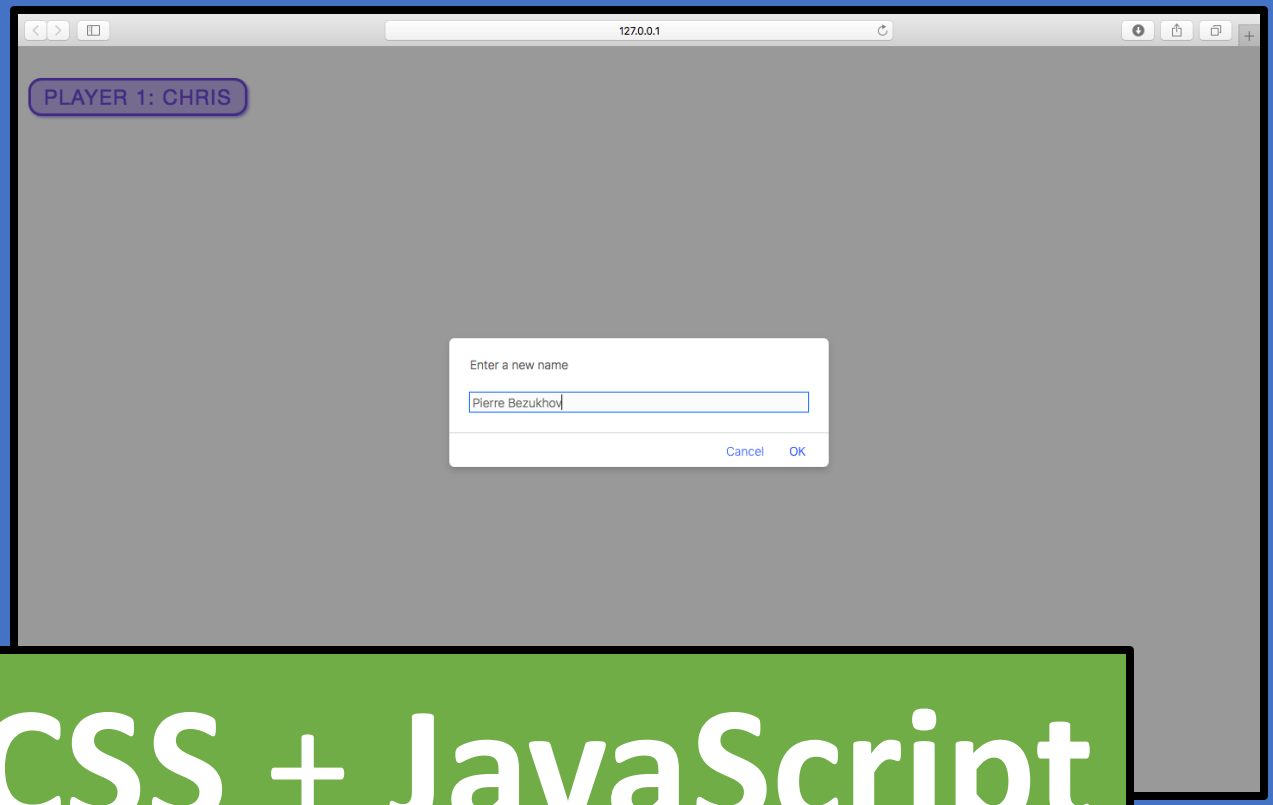

```
01  p {
02    font-family: 'helvetica neue', helvetica, sans-serif;
03    letter-spacing: 1px;
04    text-transform: uppercase;
05    text-align: center;
06    border: 2px solid rgba(0,0,200,0.6);
07    background: rgba(0,0,200,0.3);
08    color: rgba(0,0,200,0.6);
09    box-shadow: 1px 1px 2px rgba(0,0,200,0.4);
10    border-radius: 10px;
11    padding: 3px 10px;
12    display: inline-block;
13    cursor: pointer;
14  }
```

HTML + CSS

A high–level definition

- Finally, we can **add** some **JavaScript** to **implement dynamic behavior**.

```
1  const para = document.querySelector('p’);
2
3  para.addEventListener('click', updateName);
4
5  function updateName() {
6    let name = prompt('Enter a new name’);
7    para.textContent = 'Player 1: ' + name;
8  }
```

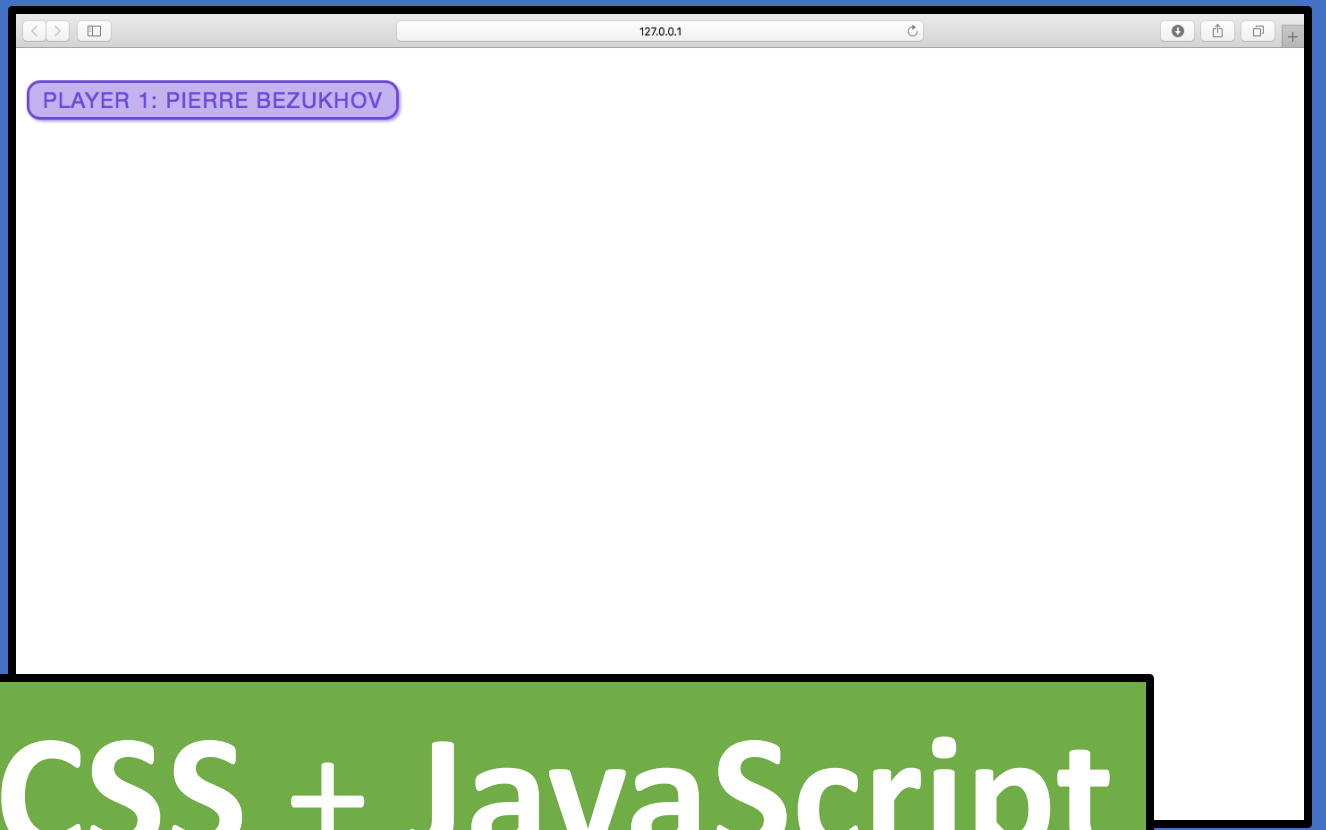


HTML + CSS + JavaScript

A high–level definition

- Finally, we can **add** some **JavaScript** to **implement dynamic behavior**.

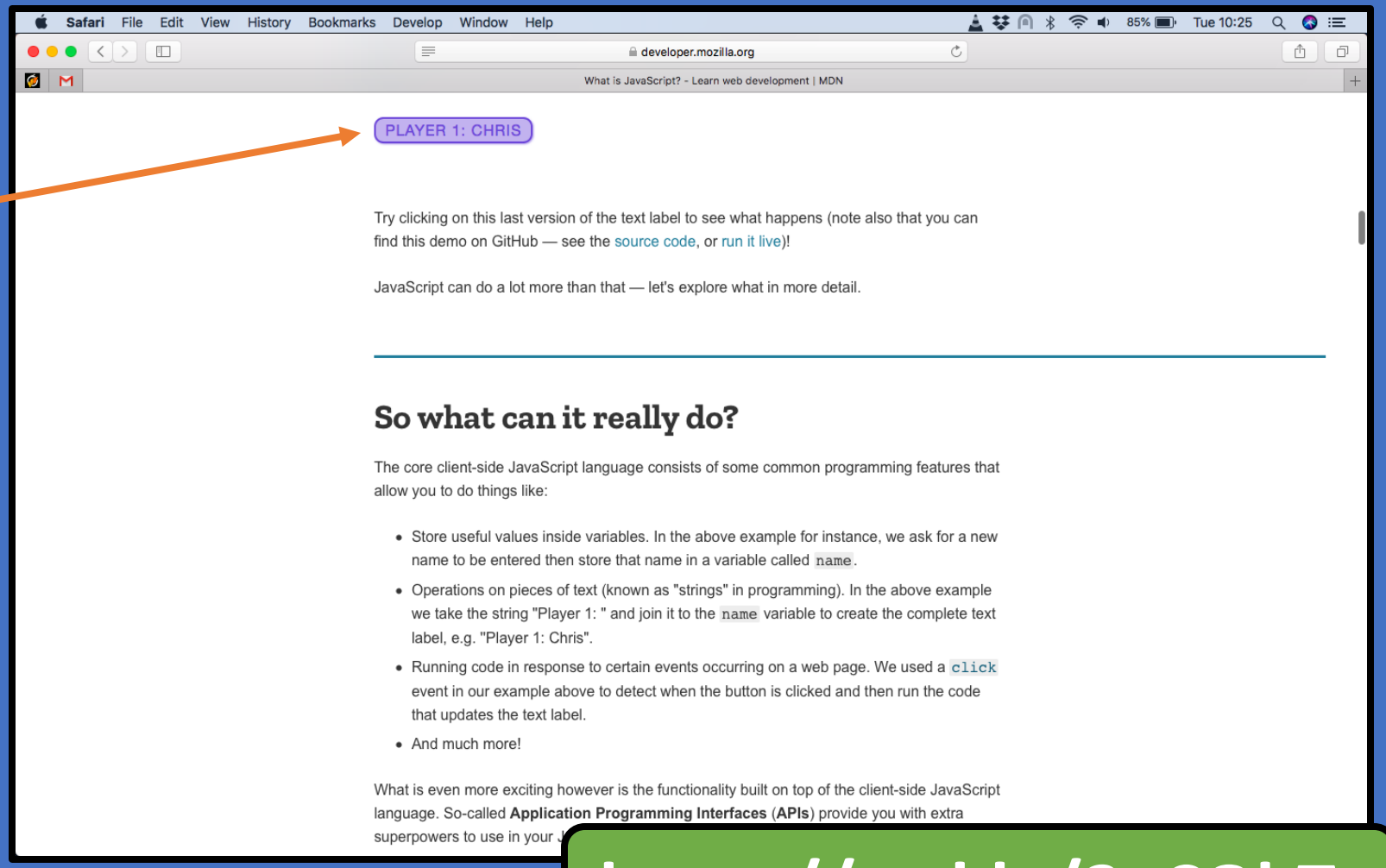
```
1  const para = document.querySelector('p’);
2
3  para.addEventListener('click', updateName);
4
5  function updateName() {
6    let name = prompt('Enter a new name’);
7    para.textContent = 'Player 1: ' + name;
8  }
```



HTML + CSS + JavaScript

A high–level definition

- You can **try** this **example online**. **Click** on this **last version in the MDN of the text label** to see **what happens**.



https://mzl.la/3eCOhTz

So what it can really do?

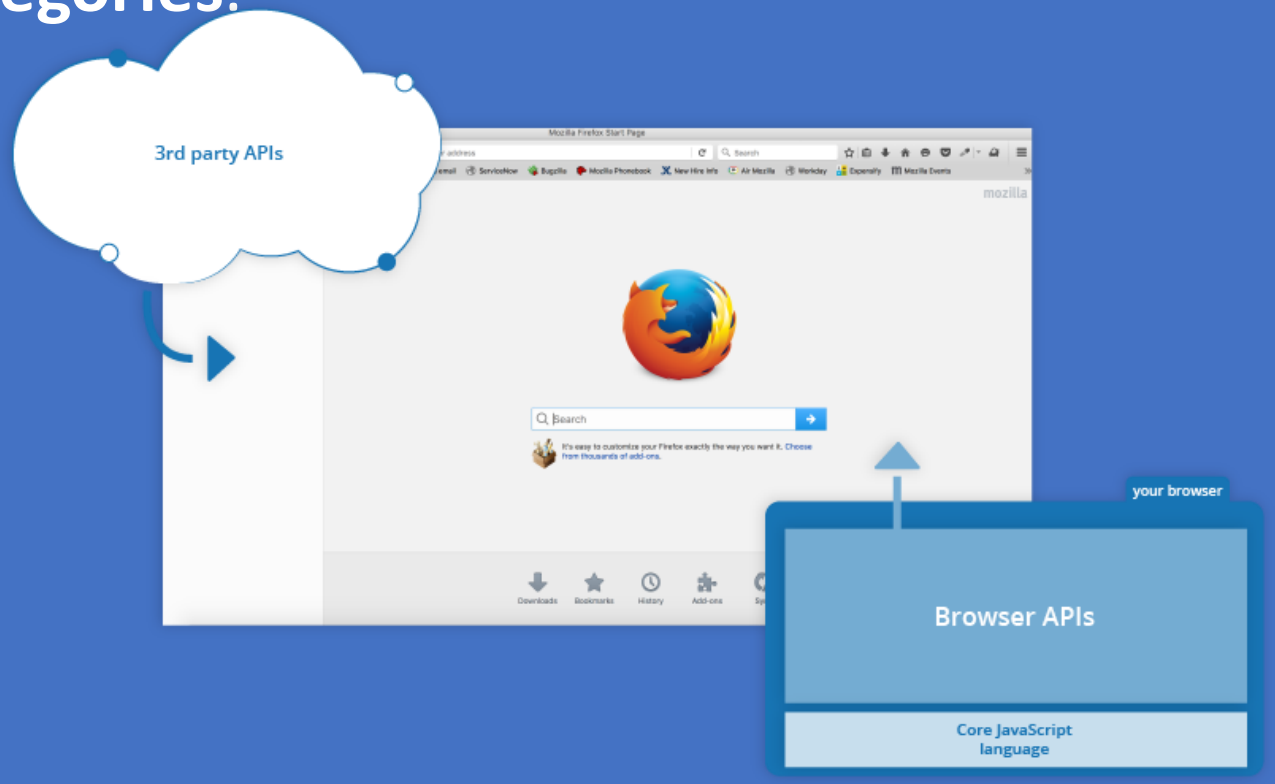
So what it can really do?

- Common **Javascript features**:
  - ✓**Store** useful **values** inside **variables**.
  - ✓**Operations** on **strings**.
  - ✓**Running code** in **response** to certain **events** occurring on a **web page**.
  - ✓…
  - ✓…

```
1  const para = document.querySelector('p’);
2
3  para.addEventListener('click', updateName);
4
5  function updateName() {
6    let name = prompt('Enter a new name’);
7    para.textContent = 'Player 1: ' + name;
8  }
```

Application Programming Interfaces - APIs

- **APIs** are **ready-made** sets of **code building blocks**.
- They generally **fall** into **two categories**.
  - ✓**Third-party** APIs.
  - ✓**Browser** APIs.



Browser APIs

- Built into your **web browser** and are **able** to **expose data** from the **surrounding computer environment**, or do **useful complex things**.

- **Examples**:
  - ✓The **DOM API** allows you to **manipulate HTML** and **CSS**.
  - ✓The **Geolocation API** retrieves **geographical information**.
  - ✓The **Canvas** and **WebGL** APIs **allow** you to **create animated 2D** and **3D graphics**.
  - ✓**Audio** and **Video APIs** allow, for instance, **play audio** and **video right** in a **web page**.

Third party APIs

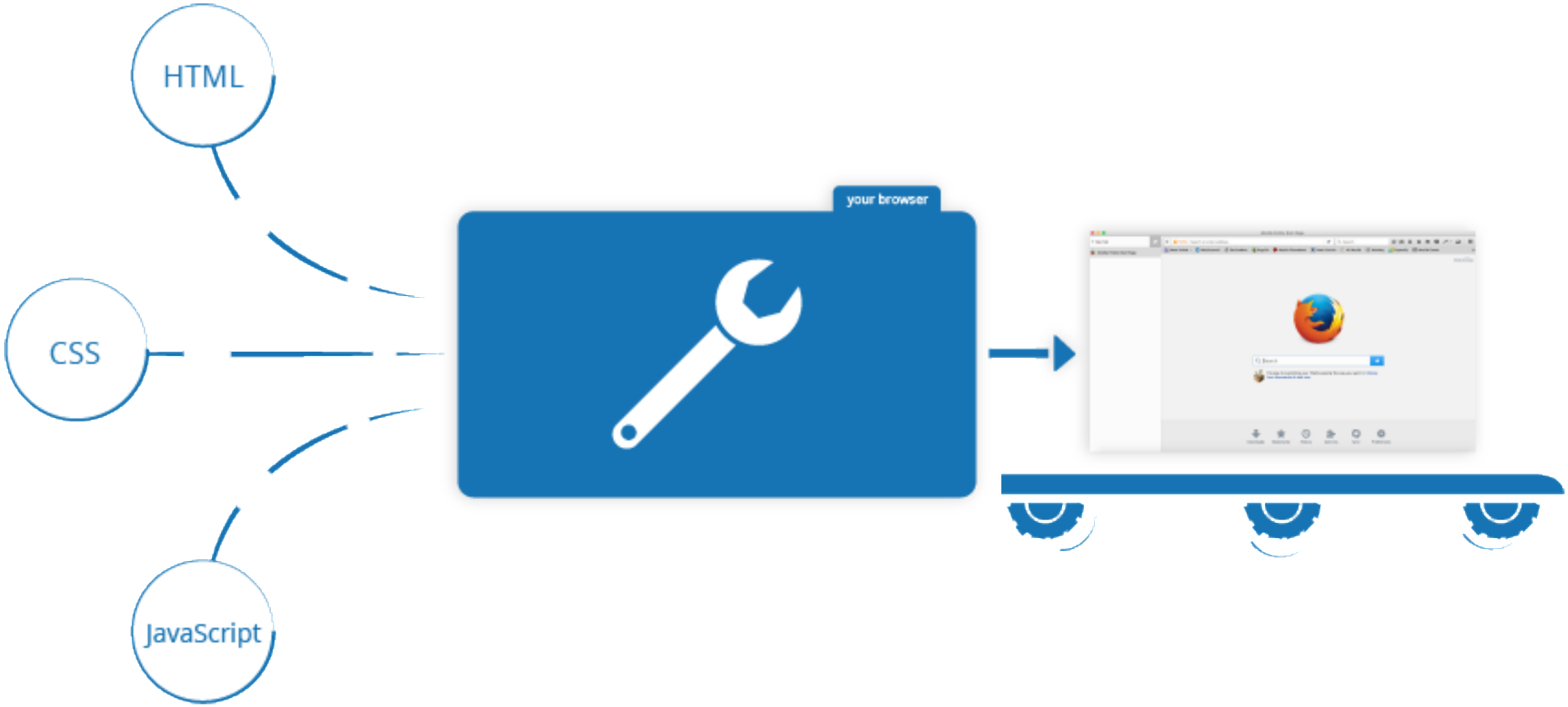
- **Not built** into the **browser** by **default**.
  - ✓You generally **have** to **grab** their **code** and **information** from **somewhere** on the **Web**.
- **Example**:
  - ✓The **Twitter API** allows you to do things like **displaying** your **latest tweets** on your **website**.
  - ✓The **Google Maps API** and **OpenStreetMap API** allows you to **embed custom maps** into your **website**, and **other such functionality**.

Note

Note: **Third-party APIs** are **advanced**, and we'll not be **covering** any of these in this **module**. You can find out much more about these in our **Client-side web APIs module**.

https://mzl.la/2ytii7V

What is JavaScript doing on your page?


HTML
CSS
JavaScript
your browser

Common use

- **Dynamically modify HTML** and **CSS** to **update** a **user interface**.
  - ✓Via the **DOM API**.
- The **code** in your **web documents** is generally **loaded** and **executed** in the **order** it **appears** on the **page**.
  - ✓If the **JavaScript loads** and **tries** to **run before** the **HTML** and **CSS** it is **affecting** has been **loaded**, **errors** can **occur**.

Browser security

- Each **browser tab** is its own **execution environment for running code**
  - ✓ This **means** that in **most cases** the **code** in **each tab** is **run completely separately**.
  - ✓ The **code** in **one tab cannot directly affect** the **code** in **another tab or** on another **website**.

Note

Note: There are **ways** to **send code** and **data between** different **websites/tabs** in a **safe manner**, but these are **advanced techniques** that we won't **cover** in this **course**.

JavaScript running order

- **When** the **browser encounters** a **block** of **JavaScript**, it generally **runs** it in **order**, from **top** to **bottom**.
  - ✓You **need** to **be careful** what **order** you **put things** in.

- **Example**
  - ✓If you **swapped** the **order** of the **first two lines of code**, it would **no longer work**.
  - ✓Instead, you'd get an **error returned** in the **Browser developer console**:
    - `TypeError: para is undefined`

```
1  const para = document.querySelector('p');
2  para.addEventListener('click', updateName);
3
4  function updateName() {
5    let name = prompt('Enter a new name');
6    para.textContent = 'Player 1: ' + name;
7  }
```

Note

Note: This is a **very common error** — you need to be **careful** that the **objects referenced** in your **code exist before** you try **to** do **stuff** to **them**.

Interpreted versus **compiled code**

Interpreted versus compiled code

- **JavaScript** is a **lightweight interpreted programming language**.
- The **web browser** receives the **JavaScript code** in its **original text form** and **runs** the **script** from that.

Interpreted versus compiled code

- Most **modern JavaScript interpreters** use a **technique** called **just-in-time compiling** to **improve performance.**
  - ✓The **JavaScript source code** gets **compiled** into a **faster**, **binary format** while the **script** is being **used**, so that it can be **run** as **quickly** as **possible**.

- However, **JavaScript** is **still considered** an **interpreted language**, since the **compilation** is **handled at run time**, **rather** than **ahead of time**.

Server-side versus client-side code

Server-side versus client-side code

- **Client-side code run** on the **user's computer**.
  - ✓When a **web page** is **viewed**, the **page's client-side code** is **downloaded**, then **run** and **displayed** by the **browser**.
- **Server-side code** on the other **hand** is **run** on the **server**.
  - ✓Its **results** are **downloaded** and **displayed** in the **browser**.
  - ✓Examples of **server-side web languages**
    - Java, PHP, Python, Ruby, ASP.NET and... **JavaScript**!

Dynamic versus static code

Dynamic versus static code

- The **word dynamic** is used to **describe** both **client-side JavaScript**, and **server-side languages**.
- **Dynamic** refers to the **ability** to **update** the **display** of a **web page/app** to **show different things** in **different circumstances**, generating **new content** as **required**.

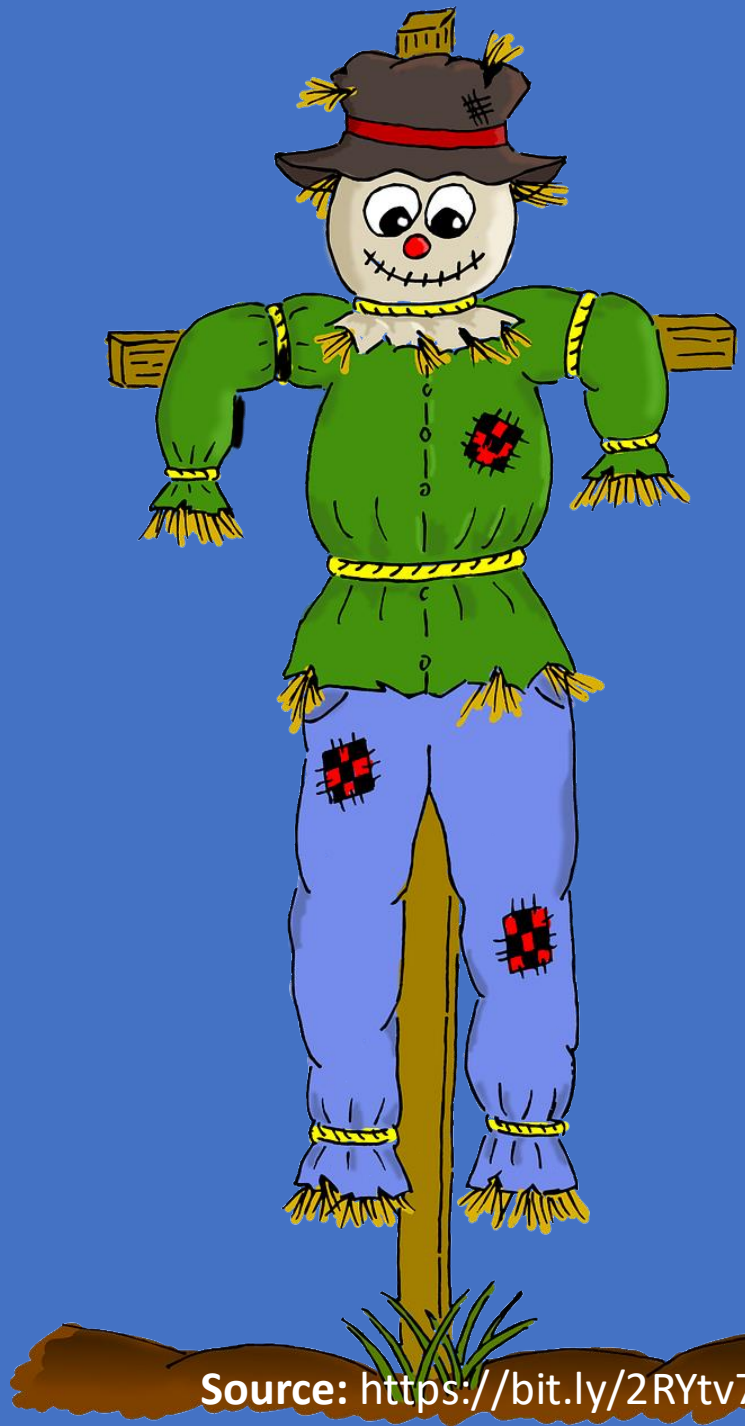
Dynamic versus static code

- **Server-side code dynamically generates new content** on the **server**,
  - ✓E.g. **Pulling data** from a **database**.

- **Client-side JavaScript dynamically generates new content** inside the **browser** on the **client**.
  - ✓E.g. **Creating** a **new HTML table**, **filling** it with **data requested** from the **server**, then **displaying** the **table** in a **web page** shown to the **user**.

Dynamic versus static code

- A **web page** with **no dynamically updating content** is **referred** to as **static**
  - ✓It **just shows** the **same con**tent all the **time**.



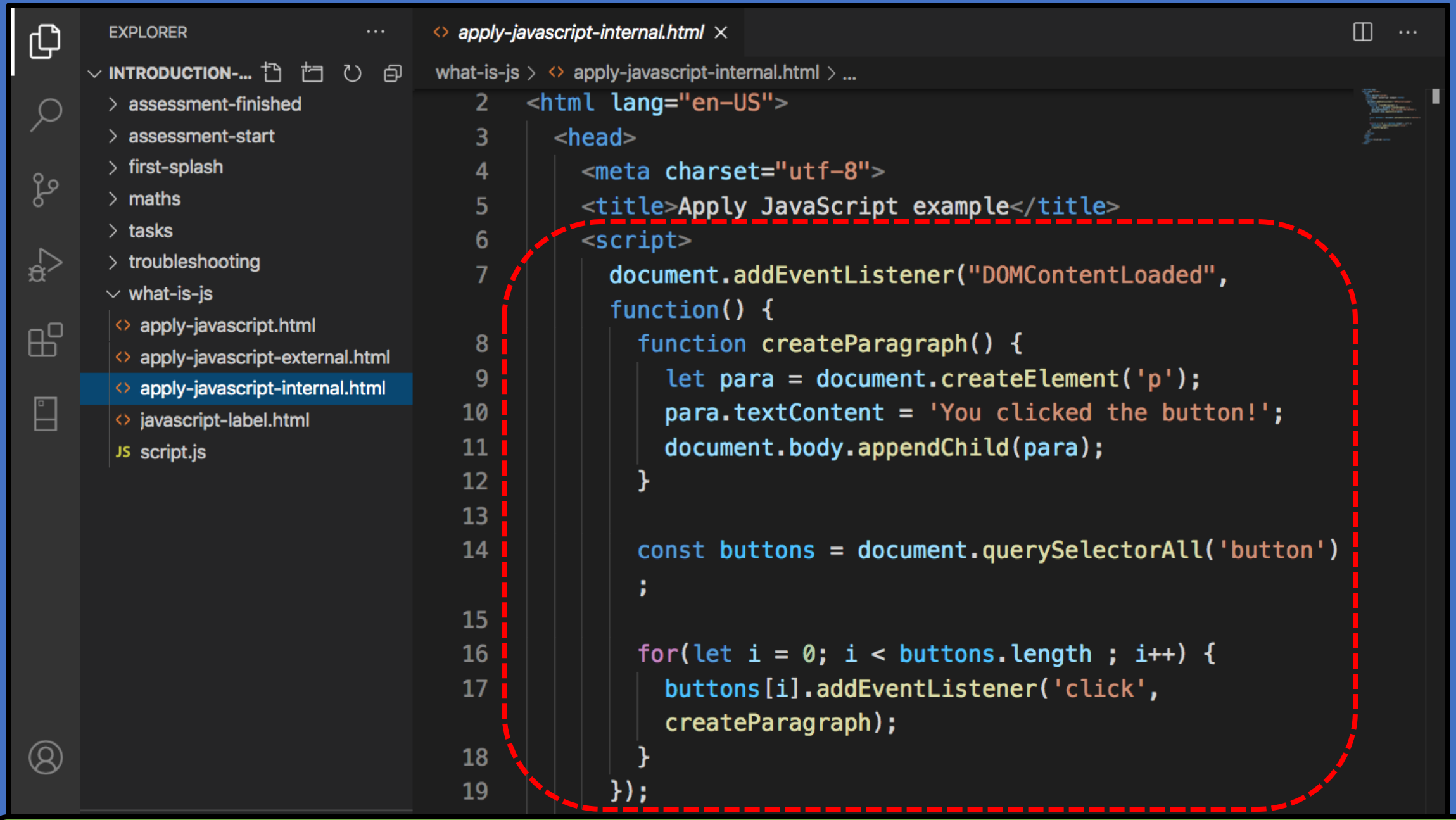
Source: https://bit.ly/2RYtv7f

How do you add JavaScript to your page?

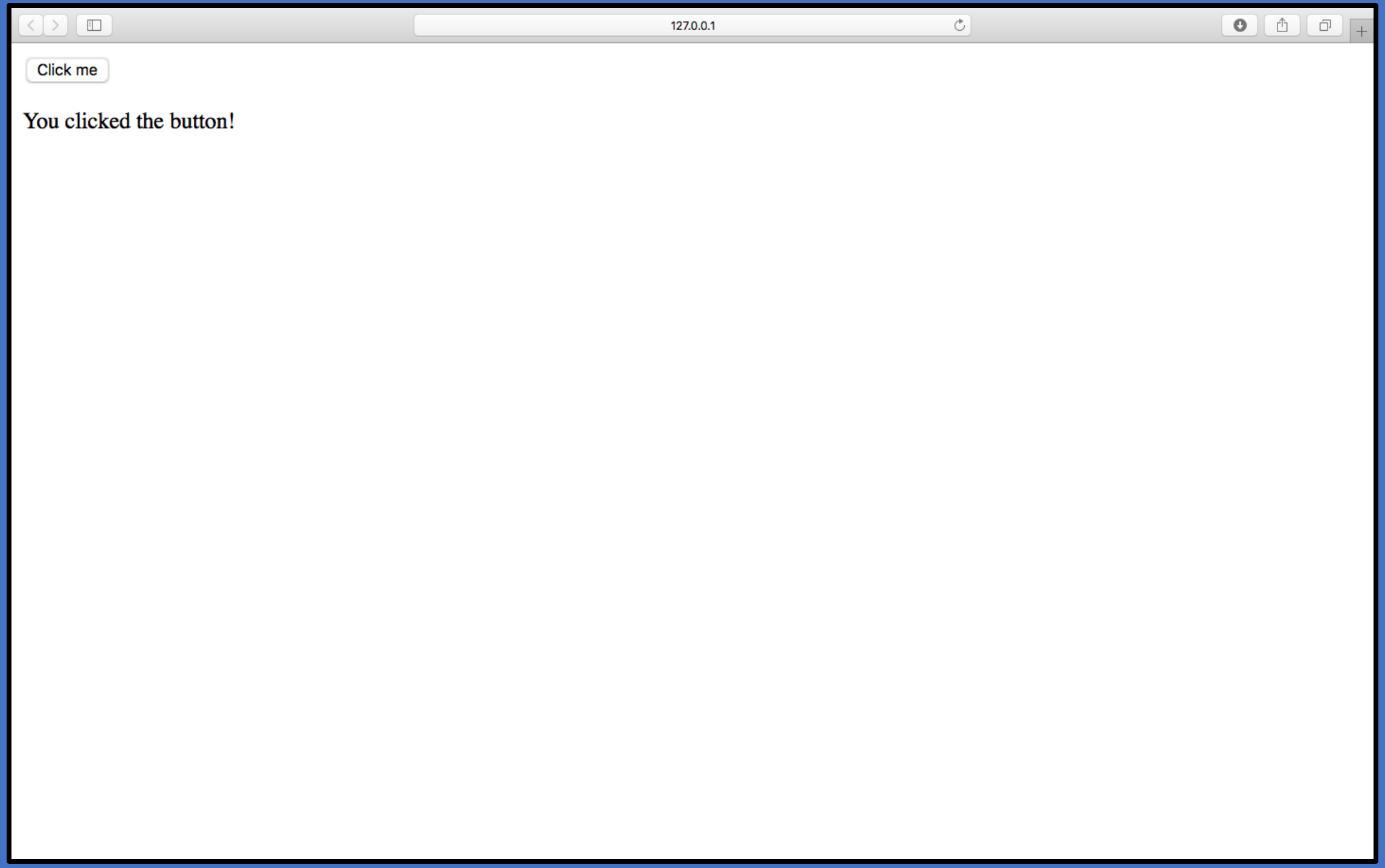
How do you add JavaScript to your page?

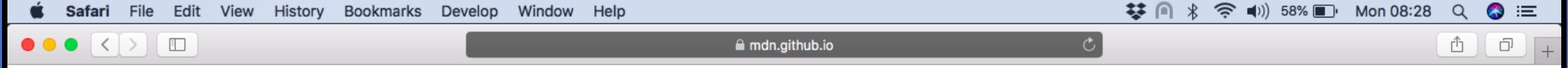
- **Similar** manner to **CSS**.
- Whereas **CSS** uses `<link>` **elements** to **apply external stylesheets** and `<style>` **elements** to **apply internal stylesheets** to **HTML**, **JavaScript** only **needs** one:
  - ✓The `<script>` **element**.

Internal JavaScript



Introduction-to-js-1/what-is-js/apply-javascript-internal.html


127.0.0.1
Click me
You clicked the button!



Click me

You clicked the button!

You can try this example online if you prefer

https://bit.ly/2xs8CtX

External JavaScript

apply-javascript-external.html — introduction-to-js-1

apply-javascript-external.html

what-is-js > apply-javascript-external.html > ...

```
1  <!DOCTYPE html>
2  <html lang="en-US">
3    <head>
4      <meta charset="utf-8">
5      <title>Apply JavaScript example</title>
6      <script src="script.js" defer></script>
7    </head>
8    <body>
9      <button>Click me</button>
10   </body>
11 </html>
12
```

script.js — introduction-to-js-1

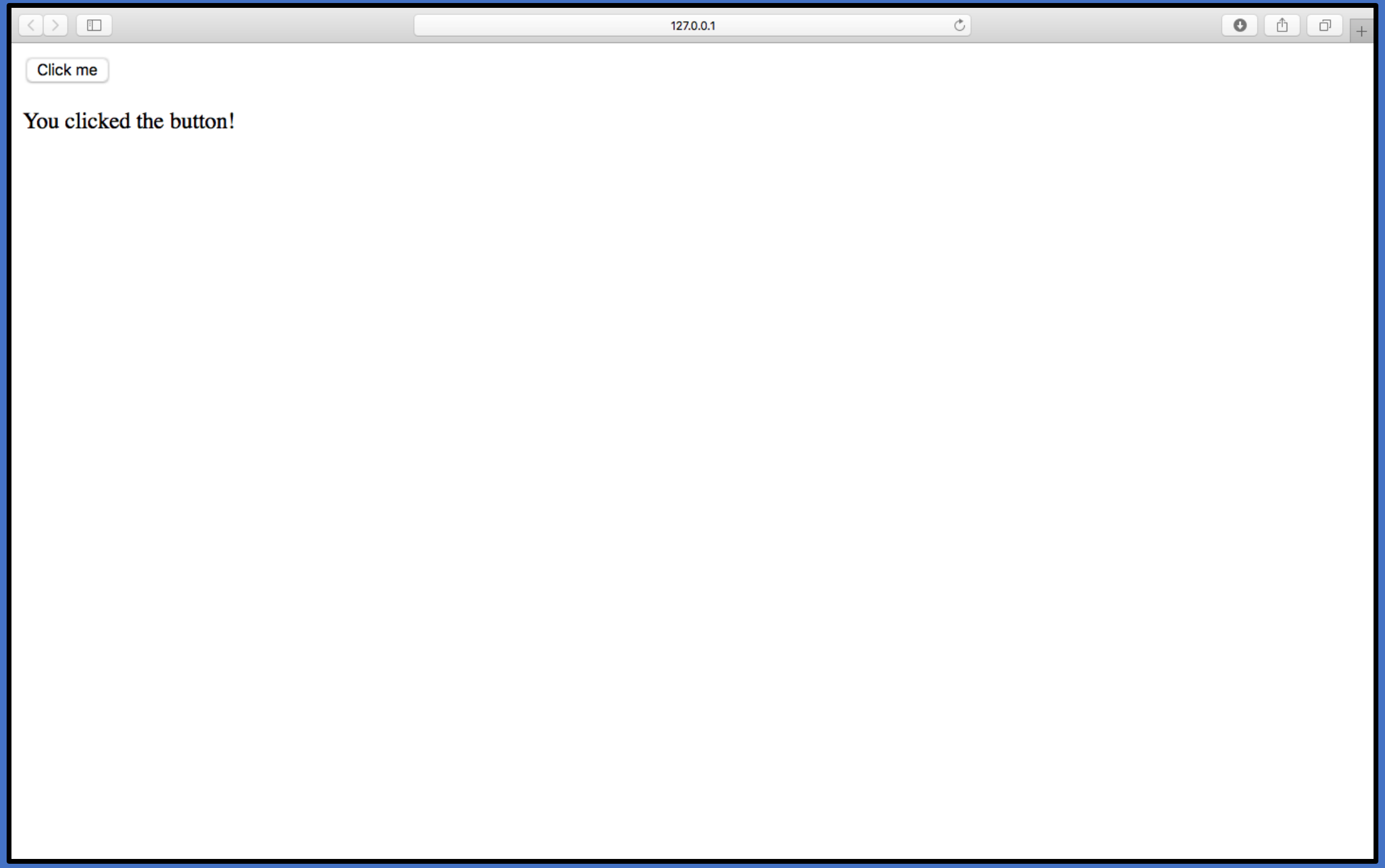
script.js

what-is-js > script.js > createParagraph

```
1  function createParagraph() {
2    let para = document.createElement('p');
3    para.textContent = 'You clicked the button!';
4    document.body.appendChild(para);
5  }
6
7  const buttons = document.querySelectorAll('button');
8
9  for(let i = 0; i < buttons.length ; i++) {
10   buttons[i].addEventListener('click',
     createParagraph);
11 }
12
```

Introduction-to-js-1/what-is-js/script.js

Introduction-to-js-1/what-is-js/apply-javascript-external.html


127.0.0.1
Click me
You clicked the button!

Inline JavaScript handlers

Inline JavaScript handlers

- **Sometimes** you'll come across **bits** of actual **JavaScript code** living **inside HTML**.

```
1  <button onclick="createParagraph()">Click me!</button>
```



```
1  function createParagraph() {
2      let para = document.createElement('p');
3      para.textContent = 'You clicked the button!';
4      document.body.appendChild(para);
5  }
```



- This **demo** has **exactly** the **same functionality** as in the **previous two sections**, except that the `<button>` **element** includes an **inline onclick handler** to make the **function run** when the **button** is **pressed**.

Inline JavaScript handlers

- **DON'T DO THIS !**
  - ✓ It is **bad practice** to **pollute HTML** with **JavaScript**
  - ✓ It is **inefficient** !
    - You'd have to **include** the `onclick="createParagraph()"` **attribute** on **every button** you **want** the **JavaScript** to apply to.

```
1  <button onclick="createParagraph()">Click me!</button>
```



```
1  function createParagraph() {
2      let para = document.createElement('p');
3      para.textContent = 'You clicked the button!';
4      document.body.appendChild(para);
5  }
```



Inline JavaScript handlers

- Using a **pure JavaScript construct** allows you to **select all** the **buttons** using **one instruction**.

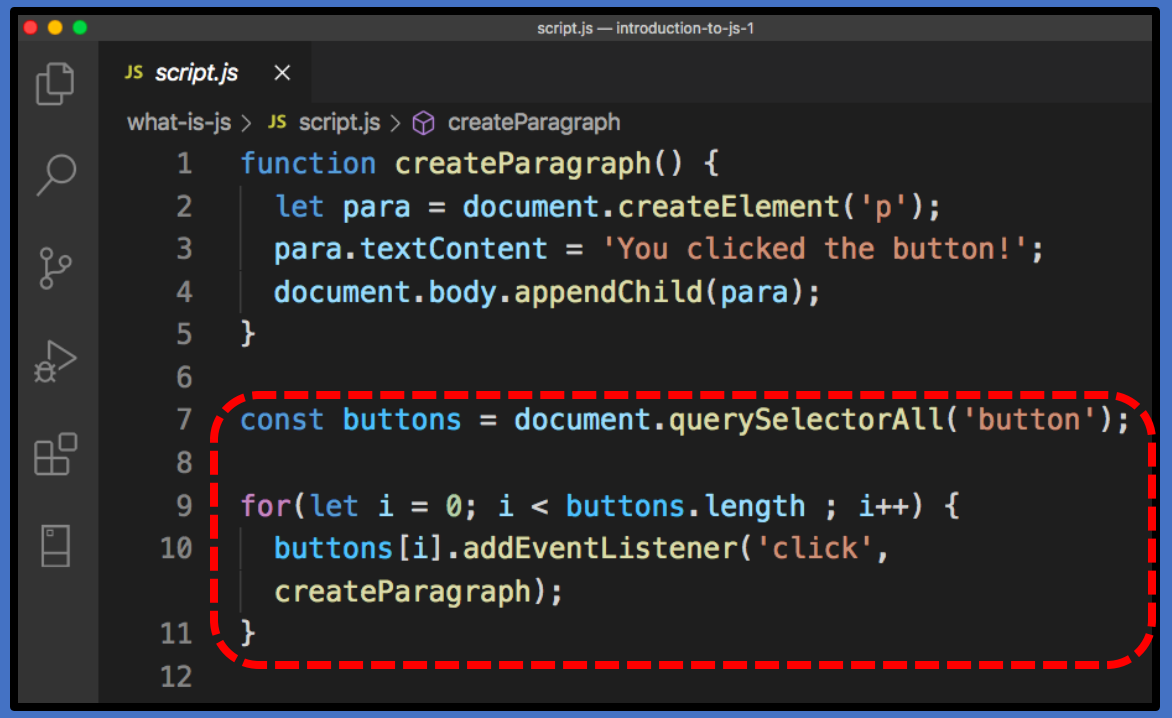
```
1  const buttons = document.querySelectorAll('button’);
2
3  for(let i = 0; i < buttons.length ; i++) {
4    buttons[i].addEventListener('click', createParagraph);
5  }
```



- **Longer** than the `onclick` **attribute**,
  - ✓but it will **work** for all **buttons**, including no matter **how many** are on the **page**, nor **how many** are **added** or **removed**.
  - ✓The **JavaScript** does **not** need to be **changed**.

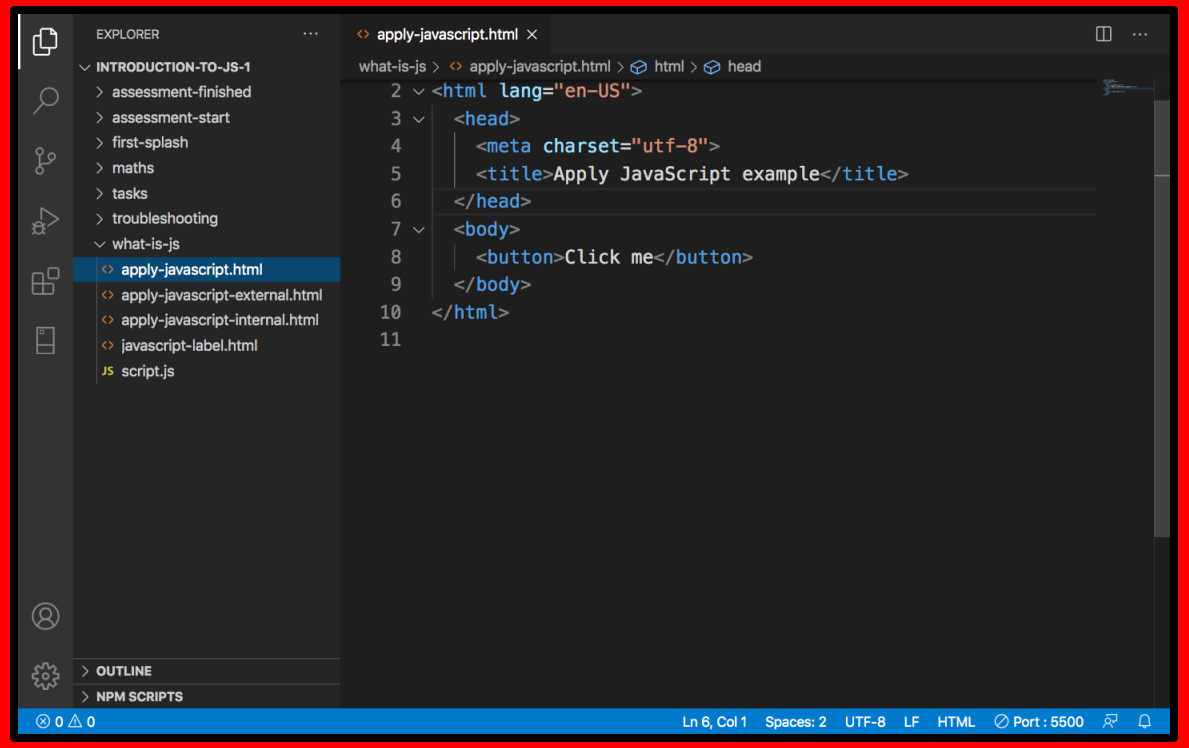
Inline JavaScript handlers

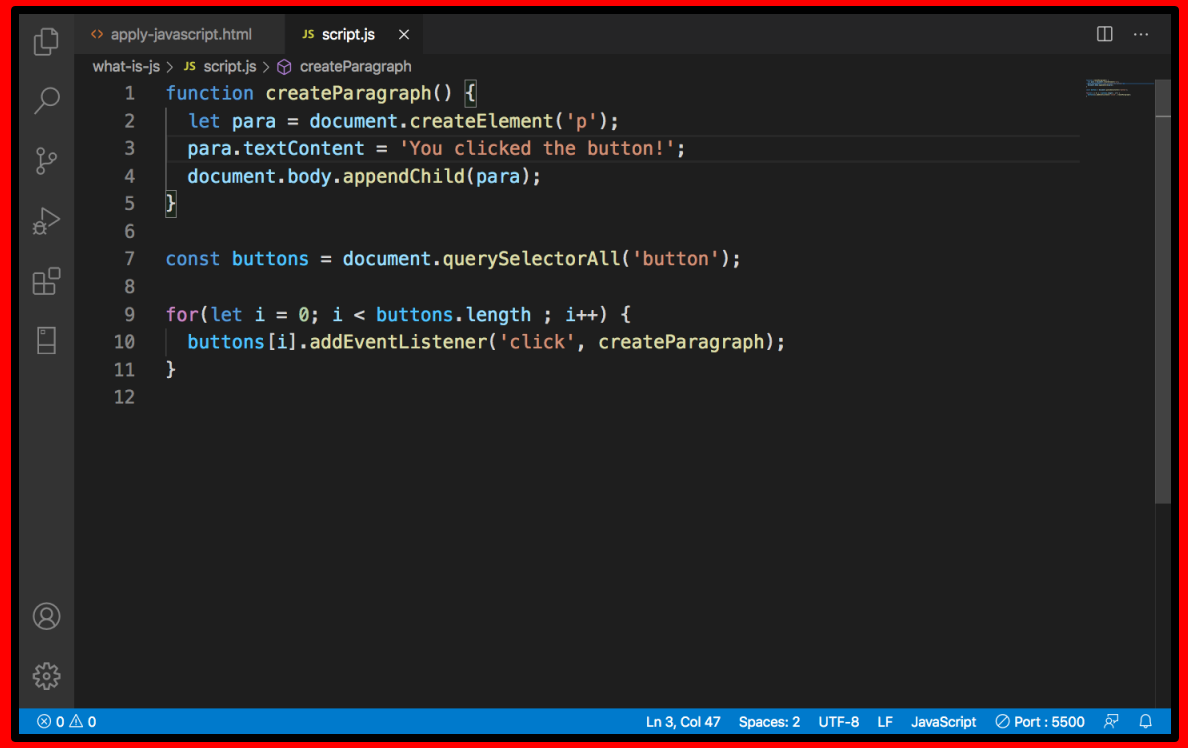
- Using a **pure JavaScript construct** allows you to **select all** the **buttons** using **one instruction**.



Active learning:
Registering **events** in **multiple buttons**

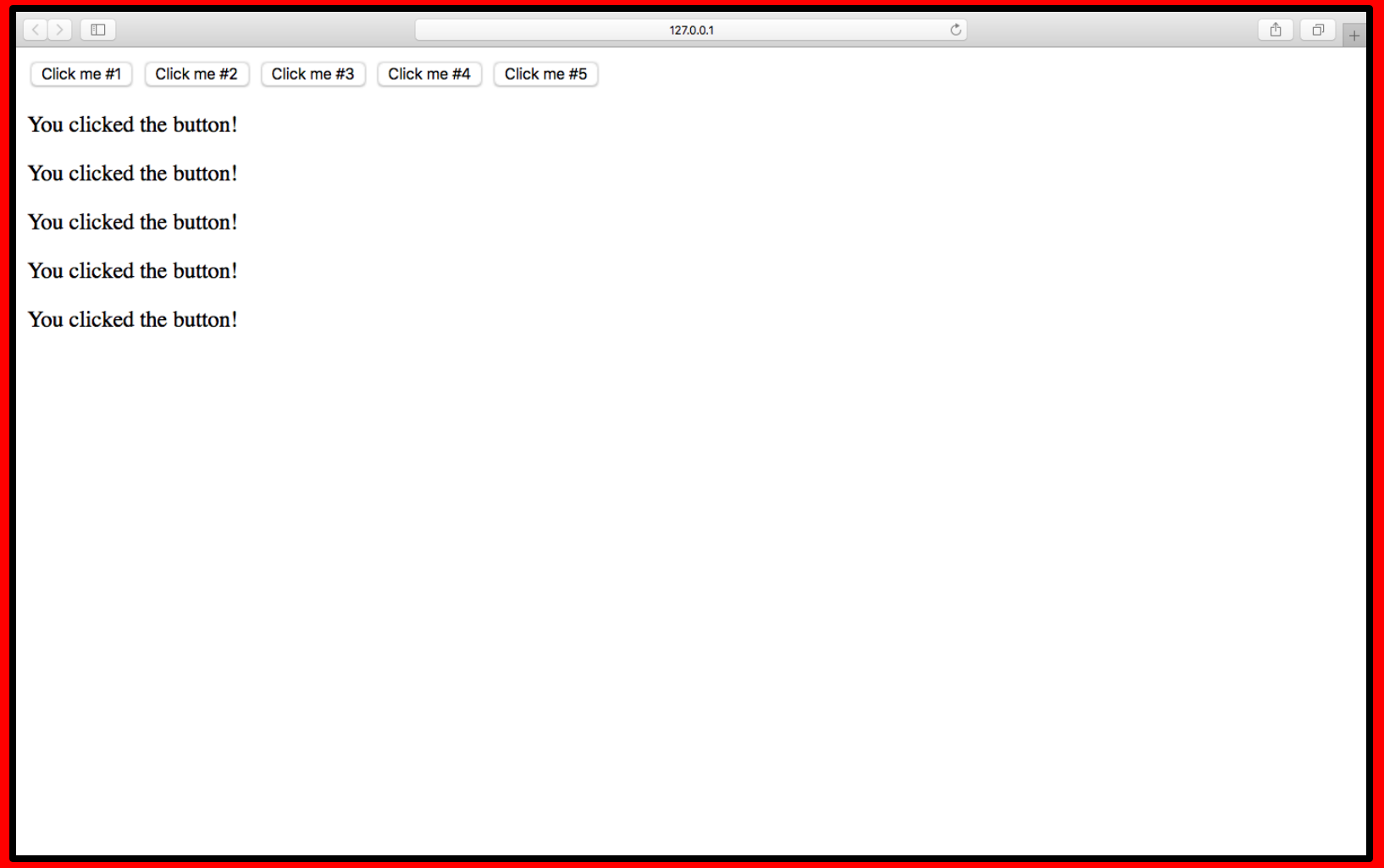
Try **editing** `apply-javascript.html`, **link** it to the `script.js` **file** and **add** some **buttons** into the file. When you **reload**, you should find that all of the **buttons** when **clicked** will **create** a **paragraph**. Neat, huh?





Introduction-to-js-1/what-is-js/apply-javascript.html

Possible solution



Script loading strategies

Script loading strategies

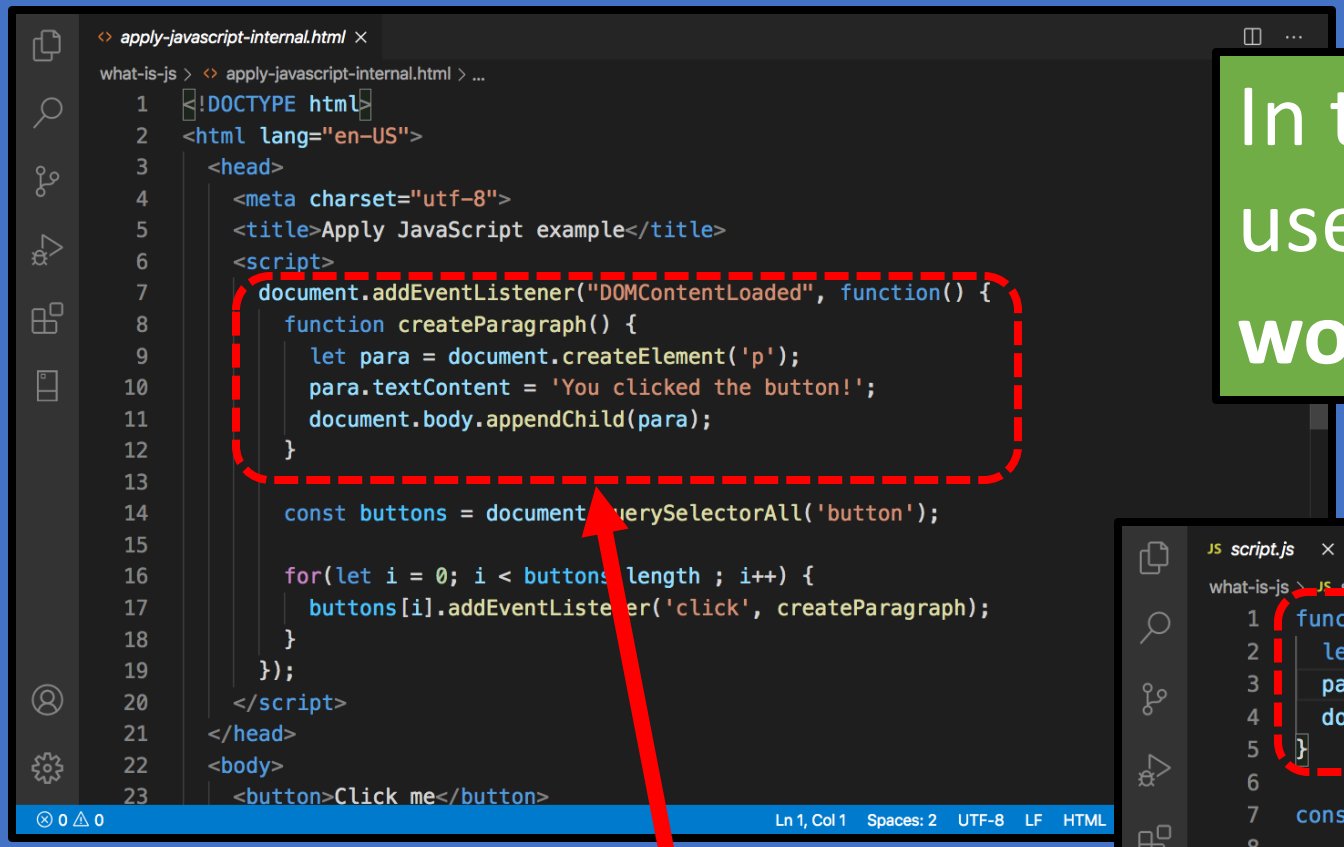
- All the **HTML** on a **page** is **loaded** in the **order** in which it **appears**.
  - ✓If you are using **JavaScript** to **manipulate** your **code**, it won't **work** if the **JavaScript** is **loaded** and **parsed before** the **HTML** you are trying to do something to.

- If **JavaScript** is **loaded** and **run** in the `head` of the **document**, this could **cause** an **error**.

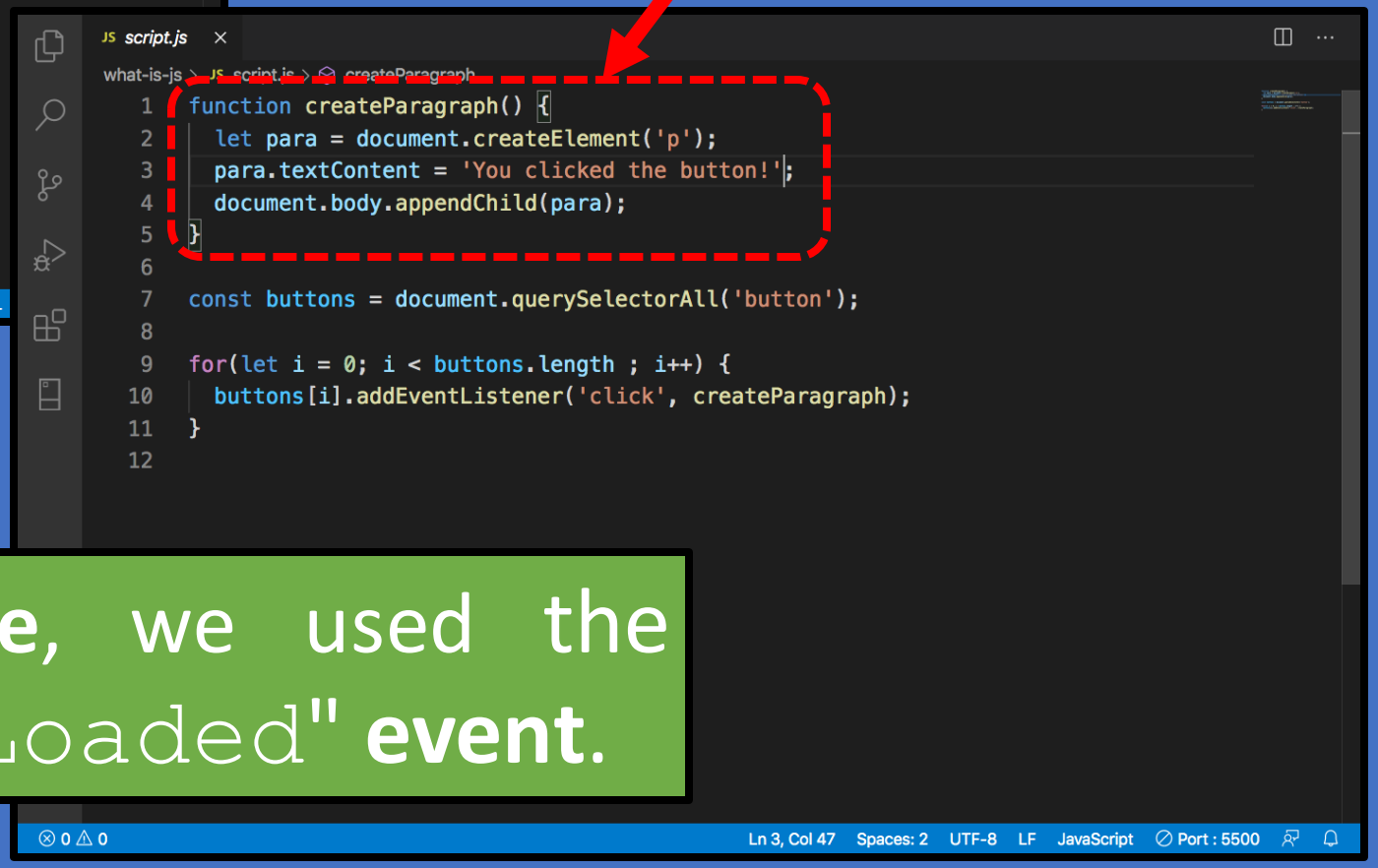
Script loading strategies

- An **event listener**, listens for the **browser's** "`DOMContentLoaded`" **event**.
  - ✓Which **signifies** that the `HTML body` is **completely loaded** and **parsed**.
- The **JavaScript** inside this **block** will **not run until after** that **event** is **fired**,
  - ✓Therefore, **possible errors** is avoided.

```
1  document.addEventListener("DOMContentLoaded", function() {
2    ...
3  });
```



In the **EXTERNAL example**, we used the **`defer` attribute**. It **only works** for **external scripts**.



In the **INTERNAL example**, we used the browser's "`DOMContentLoaded`" **event**.

Script loading strategies

- In the **external example**, we use a **more modern JavaScript feature** to **solve** the **problem**
  - ✓The `defer` **attribute**, which **tells** the **browser** to **continue downloading** the **HTML content** once the `<script>` **tag element** has been **reached**.

```
1 <script src="script.js" defer></script>
```

Note

Note: In the external case, we did not need to use the `DOMContentLoaded` event because the `defer` attribute solved the problem for us. We didn't use the `defer` solution for the internal JavaScript example because `defer` **only works for external scripts**.

Script loading strategies

- An **OLD-FASHIONED solution** to this **problem** used to be to **put** your `script` **element** right at the **bottom of the body**
  - ✓Just before the `</body>` tag), so that it would **load** after all the **HTML** has been **parsed**.

- The **problem** with this **solution** is that **loading/parsing** of the **script** is completely **blocked until** the **HTML DOM** has been **loaded**.
  - ✓On **larger sites** with **lots of JavaScript**, this can cause a **major performance issue**, **slowing down** your **site**.

The `defer` and `async` attributes as **loading strategies**

Attribute : `defer`

- **Scripts loaded** using the `defer` **attribute** will **run** in the **order** they **appear** in the **page** and **execute** them as **soon** as the **script** and **content** are **downloaded**.

```
1  <script defer src="js/vendor/jquery.js"></script>
2
3  <script defer src="js/script2.js"></script>
4
5  <script defer src="js/script3.js"></script>
```

Attribute : `async`

- **Scripts loaded** using the `async` **attribute** will **download** the **script** without **blocking rendering** the **page** and will **execute** it as **soon** as the **script finishes downloading**.

- It is **best** to **use** `async` when the **scripts** in the **page run independently** from **each other** and **depend** on **no other script** on the **page**.

Attribute : `async`

- For **example**, if you **have** the following **script elements**:

```
1  <script async src="js/vendor/jquery.js"></script>
2
3  <script async src="js/script2.js"></script>
4
5  <script async src="js/script3.js"></script>
```

Loading order not guaranteed !

- You **can't rely on the order the scripts will load** in.
  - ✓`jquery.js` **may load before** or **after** `script2.js` and `script3.js`
  - ✓In this **case**, any **functions** in those **scripts** depending on **jquery** will **produce an error.**

Attribute : `async`

- `async` should be **used** when you **have** a **bunch** of **background scripts** to **load** in and you **just want** to **get them in place** as **soon as possible**.

- **Example**:
  - ✓Maybe you have some **game data files to load**, which will be **needed** when the **game actually begins**.
  - ✓But for **now** you just **want** to **get on** with **showing** the **game intro**, **titles**, and **lobby**, **without** them **being blocked** by **script loading**.

Summary: the `async` and `defer` attributes

- `async` and `defer` both **instruct** the **browser** to **download** the **script(s)** in a **separate thread w**hile the **rest** of the page (the **DOM**, etc.) is **downloading**.

- The **page loading** is **not blocked** by the **scripts**.

Summary: the `async` and `defer` attributes

- If your **scripts** should be **run immediately** and they don't have any **dependencies**, then use async.

- If your **scripts** need to **wait** for **parsing** and **depend** on other **scripts** and/or the **DOM** being in **place**, **load** them using `defer`.
  - ✓And put their corresponding `<script>` **elements** in the **order** you **want** the **browser** to **execute** them.

Comments

Types of comments

1. A **single line comment** is **written** after a **double forward slash**.

```
1  // I am a comment
```

2. A **multi-line comment** is **written** between the **strings** /* and */

```
1  /*
2     I am also
3     a comment
4  */
```

Comments

```
01  // Function: creates a new paragraph and appends it to the bottom of the HTML body.
02
03  function createParagraph() {
04    let para = document.createElement('p’);
05    para.textContent = 'You clicked the button!’;
06    document.body.appendChild(para);
07  }
08
09  /*
10    1. Get references to all the buttons on the page in an array format.
11    2. Loop through all the buttons and add a click event listener to each one.
12
13    When any button is pressed, the createParagraph() function will be run.
14  */
15
16  const buttons = document.querySelectorAll('button’);
17
18  for (let i = 0; i < buttons.length ; i++) {
19    buttons[i].addEventListener('click', createParagraph);
20   }
```

Note

Note: In general **more comments** is usually **better** than **less**, but you should be **careful** if you find yourself adding **lots** of **comments** to **explain** what **variables** are (your **variable names** perhaps should be more **intuitive**), or to **explain** very **simple operations** (maybe your code is overcomplicated).

Questions?